Angry Bits – Code-Spickzettel

22. Oktober 2026 · Alec

Entstehung

Den Code hat Claude geschrieben, eine KI von Anthropic. Idee, Anforderungen und Entscheidungen zur Umsetzung kamen von mir: was das Spiel können soll, wie es aussieht und sich steuert, welche Änderungen nötig sind und was wieder rückgängig gemacht wird. Ich habe das Spiel im Simulator und auf der echten Lumatrix getestet und aus diesen Tests die nächsten Schritte vorgegeben, zum Beispiel eine direktere Steuerung, das Intro «ANGRY BITS» und das Löschen des Spielstands.

Überblick

Angry Bits ist ein Angry-Birds-Klon für die Lumatrix: Man schiesst Vögel mit dem Joystick auf Türme, bis alle Schweine weg sind. Auf dem Pico laufen zwei Dateien.

Datei	Inhalt
`main.py`	Das ganze Spiel: Anzeige, Steuerung, Physik, Menü
`levels.py`	Die 8 Levels als Listen von Objekten

Der Code in `main.py` ist in Abschnitte gegliedert. Jeder Abschnitt unten entspricht einem Abschnitt im Code und nennt die wichtigsten Funktionen.

Im Ordner `PC-Werkzeuge` liegen Hilfsprogramme, die nur am PC laufen und nicht zum Spiel gehören: Simulator, Level-Test, Level-Editor, Pixel-Editor und Upload-Skript.

Bildschirm

Das Spiel denkt in Koordinaten (x nach rechts, y nach oben), die LEDs sind aber eine Kette von 0 bis 63. Jedes Bild wird zuerst in einer Liste mit 64 Farben aufgebaut und dann auf einmal gesendet.

Funktion	Was sie macht
`idx(x, y)`	Rechnet eine Koordinate in die LED-Nummer um: `y * 8 + x` (unten links = 0, oben rechts = 63)
`show(buf)`	Schreibt eine Liste mit 64 Farben auf die LEDs und ruft `np.write()` auf
`draw(...)`	Baut das Spielfeld auf: Schleuder, Blöcke, Schweine, Vogel, Vorschau, übrige Vögel oben links
`color_of(kind, hp)`	Gibt die Farbe eines Objekts zurück, z.B. angeschlagener Stein dunkler

Alle Farbwerte gehen nur bis 40 statt 255, damit die LEDs nicht blenden und wenig Strom ziehen.

Steuerung

Der Joystick ist aktiv low: gedrückt bedeutet `value() == 0`. Die Pins sind wie im Cheat Sheet belegt (GPIO 2, 3, 6, 7, 8, Schiebeschalter GPIO 9).

Funktion	Was sie macht
`Button.poll()`	Meldet einen Tastendruck genau einmal. Wer hält, bekommt nach 350 ms alle 110 ms eine Wiederholung. So springt ein kurzer Druck nie zwei Stufen.
`aim(...)`	Zielen: Oben/Unten ändert den Winkel (10–80°), Links/Rechts die Kraft (7 Stufen), Mitte schiesst. Zeigt die Flugbahn-Vorschau und einen Kraftbalken.
`check_pause()`	Ist der Schiebeschalter umgelegt, blinkt ein Pause-Symbol. Mitte führt zurück ins Menü.

Physik: die Wurfparabel

Die Flugbahn wird nicht gezeichnet, sie entsteht aus drei Zeilen: Die Schwerkraft zieht bei jedem Schritt etwas von der Geschwindigkeit nach oben ab.

```
vy -= G / SUB      # Schwerkraft
fx += vx / SUB     # nach rechts
fy += vy / SUB     # nach oben oder unten
```

Funktion	Was sie macht
`velocity(angle, power)`	Zerlegt den Schuss mit Kosinus und Sinus in eine Geschwindigkeit nach rechts (`vx`) und nach oben (`vy`)
`fly(...)`	Bewegt den Vogel Schritt für Schritt und prüft nach jedem Schritt, ob auf dem Pixel ein Objekt liegt
`preview(...)`	Rechnet dieselbe Flugbahn voraus und zeigt 5 Punkte. Ist ein Hindernis im Weg, wird es rot markiert.

Die Position wird mit Kommazahlen (`fx`, `fy`) gerechnet und erst zum Zeichnen auf ein Pixel gerundet. Jeder Frame ist in 4 Teilschritte (`SUB`) zerlegt, damit ein schneller Vogel keinen Block überspringt.

Treffer und Einsturz

Jedes Objekt hat Lebenspunkte: Holz, Glas und TNT 1, Stein und Schweine 2, Plattformen sind unzerstörbar. Die Objekte liegen in einem Dictionary: Koordinate → `[Typ, Lebenspunkte]`.

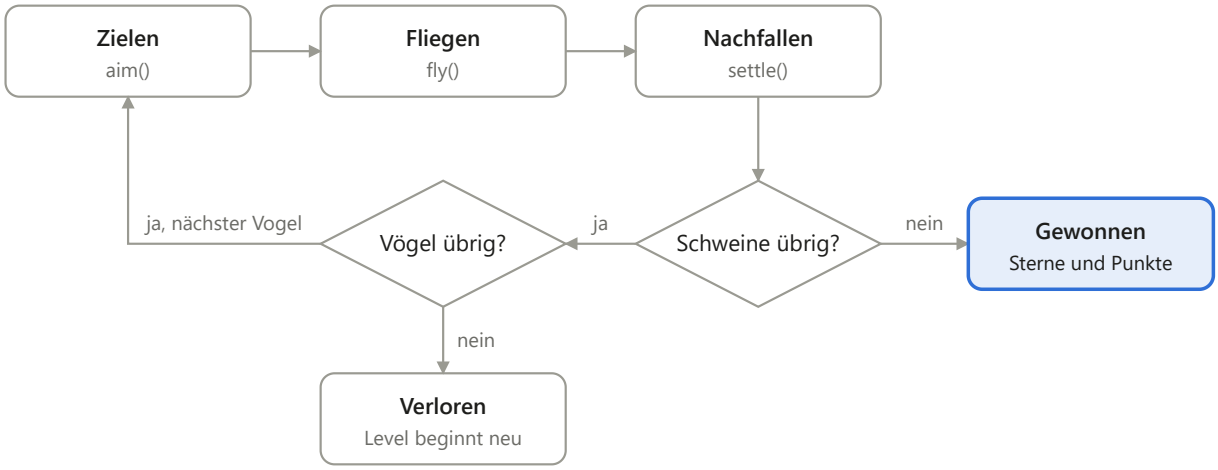
Funktion	Was sie macht
`damage(...)`	Zieht Lebenspunkte ab. Bei 0 verschwindet das Objekt und gibt Punkte.
`explode(...)`	Bombe oder TNT: Schaden im 3×3-Feld. Liegt dort weiteres TNT, explodiert es mit (Kettenreaktion).
`push(...)`	Wucht: Ein schneller Treffer schiebt das Objekt ein Feld weiter.
`settle(...)`	Nachfallen, siehe unten

So funktioniert `settle()`: Alle Objekte werden von unten nach oben geprüft. Ist das Feld darunter leer, rutscht das Objekt ein Feld nach unten. Das wiederholt sich, bis sich nichts mehr bewegt. Danach gilt: Ein Schwein, das 1 Feld fällt, ist verletzt, ab 2 Feldern ist es tot. Glas zerbricht, wenn es fällt oder wenn etwas darauf landet.

Spielablauf und Menü

Pro Vogel läuft immer dieselbe Runde: zielen, fliegen, nachfallen lassen, prüfen. Sind alle Schweine weg, ist das Level gewonnen. Sind alle Vögel verbraucht, beginnt das Level von vorn.

Pro Vogel läuft dieselbe Runde, bis alle Schweine weg sind



Funktion	Was sie macht
`main()`	Startpunkt: Intro, dann Menü und Levels in einer Endlosschleife
`menu(stars)`	Levelauswahl mit Links/Rechts. Ganz links liegt der Papierkorb zum Löschen des Spielstands.
`play_level(n)`	Spielt ein Level und gibt die Sterne zurück (0 = verloren)
`load_stars()`, `save_stars()`	Lesen und schreiben die Sterne pro Level in `save.txt` auf dem Pico

Sterne: Jedes Level hat einen Bestwert `par`, die kleinste Zahl Vögel für den Sieg. Wer mit `par` Vögeln gewinnt, bekommt 3 Sterne, mit einem mehr 2, sonst 1.

Anzeige-Extras

Text und Bilder sind als kleine Pixel-Muster im Code gespeichert: `#` bedeutet Pixel an, `.` bedeutet aus.

Funktion	Was sie macht
`text_columns()`, `scroll_text()`	Lauftext mit einer 3×5-Pixel-Schrift (`FONT`), z.B. «ANGRY BITS» und die Punktzahl
`show_bird()`	Zeigt beim Start den roten Vogel (`BIRD_ICON`), der einmal blinzelt
`win_anim()`, `lose_anim()`, `final_anim()`	Grüner Wisch mit Sternen, roter Lauftext «LOSE», Regenbogen nach dem letzten Level
`flash()`, `shake()`	Treffer blitzen kurz auf, bei Explosionen wackelt das Bild
`tone()`, `melody()`	Töne über einen optionalen Piezo-Summer (`BUZZER_PIN`). Ohne Summer wird nur gewartet.

Levels

Ein Level ist ein Dictionary in `levels.py`. Ein eigenes Level baut man, indem man einen Eintrag kopiert und die Koordinaten ändert. Die Spalten 0–2 bleiben frei für Schleuder und Vogel.

```
{"birds": "RYR", "par": 2,
 "objs": [(4, 0, "W"), (6, 0, "W"), (6, 1, "P")]}
```

Feld	Bedeutung
`birds`	Reihenfolge der Vögel: R = Rot, Y = Gelb (durchschlägt 2 Blöcke), B = Bombe
`objs`	Objekte als (x, y, Typ): W = Holz, S = Stein, G = Glas, T = TNT, F = Plattform, P = Schwein
`par`	Kleinste Zahl Vögel für den Sieg, nötig für 3 Sterne

Die Levels sind am PC geprüft: Ein Testprogramm hat für jeden Vogel alle 56 Kombinationen aus Winkel und Kraft durchgespielt. Jedes Level ist schaffbar, und die Schwierigkeit steigt von Level 1 bis 8 an.